

Rate Limiting and Abuse Protection Verification

Purpose

This testing phase focused on protecting the TEAVS–ADCRS core ingestion endpoint against excessive or repeated requests.

The endpoint under test was:

POST /api/ingestion/core

This endpoint is security-sensitive because accepted requests are forwarded into the PHOENIX core-model integration workflow. Without a request limit, an authenticated client could repeatedly submit requests at a high rate and potentially place unnecessary load on the API Gateway and downstream processing services.

Initial Review

A review of the API Gateway showed that no active rate-limiting middleware was applied to the core ingestion endpoint.

The codebase already contained the security notification type:

RATE_LIMIT_EXCEEDED

However, no active rate limiter was identified in the API Gateway route or application middleware.

This meant that authenticated and authorised clients could repeatedly call the endpoint without an application-level request threshold.

Rate-Limiting Control Implemented

A dedicated rate-limiting middleware was added to the core ingestion route.

The control was configured to allow a maximum of:

20 requests within a 60-second window per client IP address.

The updated request flow became:

JWT Authentication → RBAC → Rate Limiting → Input Validation → Core Integration Controller

The protected route therefore applies multiple controls before data is accepted into the core integration workflow.

Runtime Abuse Test

A controlled burst test was performed against:

POST /api/ingestion/core

using a valid ingestion_service JWT and a valid TEAVS-ADCRS payload.

Twenty-five requests were submitted in rapid succession.

The initial requests were accepted with:

HTTP 202 Accepted

Once the configured request threshold was reached, subsequent requests were rejected with:

HTTP 429 Too Many Requests

This confirmed that the rate limiter was active and prevented continued request submission after the configured threshold was exceeded.

Test Result

Test	Expected Result	Actual Result	Status
Requests below threshold	HTTP 202	HTTP 202	PASS
Requests after threshold	HTTP 429	HTTP 429	PASS

Security Impact

The rate-limiting control reduces the risk of excessive request submission against the TEAVS-ADCRS integration endpoint.

This helps protect:

- the API Gateway
- the RabbitMQ integration workflow

- downstream core-model processing
- backend service availability

The control also provides an additional layer of protection even when a client possesses a valid JWT and an authorised role.

Evidence

EVID-RATE-01 – Core ingestion rate-limit enforcement

```

● (base) abdulrehman@Abduls-Air backend % for i in {1..25}; do
  code=$(curl -s -o /dev/null -w "%{http_code}" \
    -X POST http://localhost:3001/api/ingestion/core \
    -H "Authorization: Bearer $INGESTION_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{
      "url": "https://example.com/teavs-alert",
      "text": "TEAVS detected a potential phishing hazard",
      "timestamp": "2026-09-27T08:00:00+10:00",
      "hazard_type": "phishing",
      "hazard_severity": 4,
      "hazard_timestamp": "2026-09-27T07:59:00+10:00",
      "hazard_location": "VIC",
      "hazard_status": "active",
      "alert_level": "critical",
      "source": "TEAVS"
    }')
  echo "Request $i -> HTTP $code"
done
Request 1 -> HTTP 202
Request 2 -> HTTP 202
Request 3 -> HTTP 202
Request 4 -> HTTP 202
Request 5 -> HTTP 202
Request 6 -> HTTP 202
Request 7 -> HTTP 202
Request 8 -> HTTP 202
Request 9 -> HTTP 202
Request 10 -> HTTP 202
Request 11 -> HTTP 202
Request 12 -> HTTP 202
Request 13 -> HTTP 202
Request 14 -> HTTP 202
Request 15 -> HTTP 202
Request 16 -> HTTP 202
Request 17 -> HTTP 202
Request 18 -> HTTP 202
Request 19 -> HTTP 202
Request 20 -> HTTP 429
Request 21 -> HTTP 429
Request 22 -> HTTP 429
Request 23 -> HTTP 429
Request 24 -> HTTP 429
Request 25 -> HTTP 429
○ (base) abdulrehman@Abduls-Air backend %

```

Current Limitation

The current implementation uses an in-memory request counter.

This is suitable for the current local development and verification environment.

For a distributed or production deployment with multiple API Gateway instances, a shared rate-limiting store such as Redis would provide more consistent enforcement across instances.